Longest Common Substring

The longest common substring problem is the problem of finding the longest string (or strings) that is a substring (or are substrings) of two strings.

Naive solution would be to consider all substrings of the second string and find the longest substring that is also a substring of first string. The time complexity of this solution would be `O((m+n)*m)` as it takes `(m+n)` time for substring search and there are m^2 substrings of second string.

The idea is to find the longest common suffix for all pairs of prefixes of the strings using Dynamic Programming using the relation –

```
LCSuffix [ i ][ j ] = | LCSuffix [ i-1 ][ j-1 ] + 1    (if X[ i ] == Y[ j ])
                      | 0                              (otherwise)
```

For example, consider strings `'ABAB'` and `'BABA'`.

		A	B	A	B
	0	0	0	0	0
B	0	0	1	0	1
A	0	1	0	2	0
B	0	0	2	0	3
A	0	1	0	3	0

Finally, the length of the longest common substring would be the maximal of these longest common suffixes of all possible prefixes.

Why do we take [i-1][j-1] only? Because a substring will be contiguous. If [i-1][j-1] matched then [i][j] = 1 + [i-1][j-1] where [i-1][j-1] != 0

else [i][j] = 1 + [i-1][j-1] where [i-1][j-1]=0

```
vector<string> LCSubstring(string X, string Y){

     int M = X.length();
     int N = Y.length();

     int DP[M][N];

     vector<string> ans;
     int maxLCS = -1;

     for(int i=0; i<M; i++){
         for(int j=0; j<N; j++){

               if(X[i]==Y[j]){
                  if(i==0 || j==0)    DP[i][j] = 1;
                  else
                       DP[i][j] = DP[i-1][j-1] + 1;

                 //check for max condition
                  if(DP[i][j]>maxLCS){
                        //create a new vector
                         maxLCS = DP[i][j];
                         vector<string>temp;
                         ans = temp;
                         ans.push_back(X.substr(i-maxLCS+1,maxLCS));
                  }
                  else if(DP[i][j]==maxLCS)
                  {
                         ans.push_back(X.substr(i-maxLCS+1,maxLCS));
                  }
                 }
                 else{
                      DP[i][j] = 0;
                 }
            }
       }
       return ans;
}
```